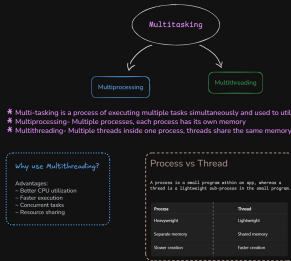


MULTITHREADING



LIFECYCLE OF THREADS:

The thread lifecycle describes the different states a thread goes through during execution.



New (Newborn) – A thread is in the new state when the thread object is created BUT the start() method has not been called yet.

Runnable – A thread enters the runnable state after the start() method is called and it becomes ready to run, waiting for CPU scheduling.

Running – A thread is in the running state when the CPU allocates time to it and the run() method is being executed.

Blocked – A thread tries to enter the synchronized block or method.

Waiting – When a thread waits for a specific condition to be met before continuing its executing, in this notify() or notifyAll() gets called.

Timed Waiting – A thread waiting for the completion of other threads before proceeding, in this sleep() or join() gets called.

Terminated (Dead) – A thread is in the terminated state after the run() method has completed or the thread has stopped executing.

CREATION OF USER-DEFINED THREADS

By extending Thread Class:

A thread can be created by extending the Thread class and overriding the run() method.

```
import java.util.*;

class Example extends Thread {
    public void run() {
        System.out.println("Using Thread Class");
    }
}

public static void main(String[] args) {
    Example t1 = new Example();
    t1.start();
}
```

By implementing Runnable interface:

A thread can be created by implementing the Runnable interface and overriding the run() method.

```
import java.util.*;

class Example implements Runnable {
    public void run() {
        System.out.println("Using Runnable Interface");
    }
}

public static void main(String[] args) {
    Runnable r1 = new Example();
    Thread t1 = new Thread(r1);
    t1.start();
}
```

The Important Notes:

- start() internally calls run() and executes the code present in it.
- The run() method is a synchronized method, so it is not safe to use start() or run() method in a thread.
- The run() method is a synchronized method, so it is not safe to use start() or run() method in a thread.
- The run() method is a synchronized method, so it is not safe to use start() or run() method in a thread.

Methods of Thread class:

start() – Starts a new thread and internally calls the run() method to execute the thread code.

run() – Contains the code that defines the task to be performed by the thread.

join() – Makes the current thread wait until the specified thread finishes its execution.

getPriority() – Returns the priority of the thread.

setPriority() – Sets the priority of the thread to control its scheduling preference.

getName() – Returns the name of the thread.

setName() – Changes or assigns a new name to the thread.

getId() – Returns the unique ID assigned to the thread by the JVM.

setId() – Used to assign an ID to a thread. (Note: In actual Java, thread ID is automatically assigned and cannot normally be set.)

isAlive() – Checks whether the thread is still running or has finished execution.

yield() – Temporarily passes the current thread and allows other threads of the same priority to execute.

suspend() – Temporarily pauses the execution of a thread. (Deprecated in Java.)

resume() – Restarts a thread that was previously suspended. (Deprecated in Java.)

stop() – Stops the execution of a thread immediately. (Deprecated and unsafe.)

destroy() – Destroys the thread without executing it. (Deprecated and rarely used.)

notify() – Wakes up a single thread that is waiting on the object's monitor.

P.S: suspend(), resume(), stop(), and destroy() are deprecated methods because they can cause deadlocks and unsafe thread behavior.

sleep() – pauses the execution of the current thread for a specified time (in milliseconds), used to create delays and control the execution speed of threads.

Common Uses:

- creating delay between operations
- simulating real-time waiting (like countdowns)
- preventing continuous CPU usage by a thread

```
class Example extends Thread {
    public void run() {
        for(int i = 1; i <= 5; i++) {
            System.out.println(i);
            try {
                Thread.sleep(1000); // pauses thread for 1 second
            } catch (Exception e) {}
        }
    }

    public static void main(String args[]) {
        Example t1 = new Example();
        t1.start();
    }
}
```

THREAD PRIORITIES:

Thread Priority determines the importance of a thread for CPU scheduling. Threads with higher priority are more likely to be executed before lower priority threads by the JVM scheduler.

Priority values:

Priority Value	Constant	Meaning
1	MIN_PRIORITY	Lowest priority
5	NORM_PRIORITY	Default priority
10	MAX_PRIORITY	Highest priority

By default, every thread has priority 5.

Methods Used:

- setPriority(int priority) – sets the priority of a thread.
- getPriority() – returns the priority of a thread.
- Thread priority helps the JVM scheduler decide which thread should run first.
- A thread with higher priority may get CPU earlier than a lower priority thread.
- Priority does not guarantee execution order because scheduling depends on the OS and JVM.
- Child threads usually inherit the priority of the parent thread.
- The priority value must be between 1 and 10, otherwise Java throws an exception.

```
Implementation:
class Example extends Thread {
    public void run() {
        System.out.println(Thread.currentThread().getName() +
            " Priority: " + Thread.currentThread().getPriority());
    }
}

public static void main(String[] args) {
    Example t1 = new Example(); //Thread 1
    Example t2 = new Example(); //Thread 2
    t1.setPriority(Thread.MAX_PRIORITY);
    t2.setPriority(Thread.MIN_PRIORITY);
    t1.start();
    t2.start();
}
```

SUPPOSE:

In a bank account with ₹1000.
Two people, Rahul and Riya, try to withdraw ₹700 each at the same time from two ATMs.

Rahul checks balance = ₹1000
Riya checks balance = ₹1000

Both think "Enough money hai" and withdraw ₹700.
Now total withdraw = ₹1400, even though the account had only ₹1000.

This happens because both transactions happened at the same time without coordination.

This is called a Race Condition.

A race condition occurs when multiple threads access and modify shared data simultaneously, causing incorrect results.

To prevent this, we must ensure only one thread accesses the shared resource at a time.
Control mechanism is called Synchronization.

SOME OTHER FEATURES:

Inter-thread communication is a mechanism that allows multiple threads to communicate and coordinate with each other while sharing resources. It helps threads avoid busy waiting and work in a synchronized way.

Methods Used:

- wait() – causes the current thread to wait until another thread notifies it.
- notify() – wakes up one waiting thread.
- notifyAll() – wakes up all waiting threads.

Points to remember:

- These methods are defined in the Object class.
- They must be used inside a synchronized block or method.
- wait() releases the lock of the object while waiting.
- notify() wakes one thread, while notifyAll() wakes all waiting threads.

Commonly used in Producer-Consumer problems.

Example:

```
class Test {
    synchronized void print() {
        try {
            wait();
        } catch (Exception e) {}
        System.out.println("Thread resumed");
    }
}
```

Deadlock is a situation where two or more threads are waiting for each other to release resources, and none of them can proceed.

Occurs when threads hold resources and wait indefinitely for other resources.

- No thread can continue execution.
- It usually happens when multiple locks are used improperly.
- It causes the program to freeze or hang.
- Avoided by proper lock ordering or reducing synchronized blocks.

Example:

Thread 1 locks Object A and waits for Object B
Thread 2 locks Object B and waits for Object A

Both threads keep waiting → Deadlock

A Daemon Thread is a background thread that provides services to user threads and automatically stops when all user threads finish execution.

Examples:

- Garbage Collector
- Background monitoring tasks

- Runs in the background to support other threads.
- JVM automatically terminates daemon threads when user threads end.
- It must be set before starting the thread.

Method used: setDaemon(true) and isDaemon()

Example:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Daemon thread running");
    }
}

public static void main(String[] args) {
    MyThread t = new MyThread();
    t.setDaemon(true);
    t.start();
}
```

SYNCHRONIZATION:

It is a technique through which we can control multiple threads from accessing a shared resource at the same time. Only one thread can enter inside the synchronized area at a time.

It is classified into three categories:

- Method-level Synchronization- entire method becomes locked for one thread at a time, 'synchronized' keyword used before method. Only one thread can execute the method at a time. Lock is applied on the object.
- Block-level Synchronization- only specific critical section is synchronized instead of the entire method. Uses synchronized() block. More efficient than method synchronization since only the critical section is locked, not the entire method.
- Static-level Synchronization- used when static data is shared across all objects. Remember that here, lock is applied to the class instead of object.

Only one thread across all objects can access it.

SUMMING UP:

Type	Lock Applied To	Use Case
Method-level	Method	Entire method needs protection
Block-level	Specific object	Only critical code needs lock
Static-level	Class	Shared static resources